Universidade de Brasília (UnB)
Departamento de Ciência da Computação

Design and Complexity of Algorithms
Professor Marcelo Antonio Marotta
Final Project

Learning-Path Optimization as a 0/1 Knapsack Problem

Dynamic Programming versus the Exhaustive DFS of RUTICO and Greedy Selection

Problem:	Selecting the set of learning modules that maximizes the learning score within a study-time budget (0/1 Knapsack Problem)
Article studied:	RUTICO: Recommending Successful Learning Paths Under Time Constraints (UMAP 2017)
Algorithms:	Exhaustive DFS (authors), Dynamic Programming (proposal), Greedy by score/time ratio (comparison), plus Branch and Bound, Meet-in-the-Middle and FPTAS

Group members

[Member Name 1]	[Student ID 1]
[Member Name 2]	[Student ID 2]
[Member Name 3]	[Student ID 3]
[Member Name 4]	[Student ID 4]

Brasília, October 2026

Abstract

Choosing which learning modules a student should study within a limited amount of time is a combinatorial optimization problem that arises in e-learning recommender systems. We model it as the 0/1 Knapsack Problem and study it through the article *RUTICO: Recommending Successful Learning Paths Under Time Constraints*, whose solution enumerates the feasible paths of a course graph with a depth-first search (DFS). We analyze the design and the complexity of three algorithmic strategies: the exhaustive DFS with time pruning used by the authors, a Dynamic Programming (DP) algorithm that we propose, and a greedy heuristic based on the score/time ratio. Three further algorithms (Branch and Bound, Meet-in-the-Middle and an FPTAS) are used as additional points of comparison. We prove the correctness and derive the best-, worst- and average-case behavior of each algorithm, and we validate the analysis with a hand-written Python implementation and with experiments that count elementary operations, so that conclusions do not depend on the hardware. The DFS generates exactly $2^{n+1}-1$ nodes in its worst case, whereas the DP performs exactly $n(T+1)$ operations on any instance and is pseudo-polynomial; the greedy heuristic is fast but loses up to 54.5% of the optimal score on our small random instances and 99.8% on an adversarial instance. We show that DP is the most suitable algorithm for the learning-path scenario, since it is exact, has a predictable cost, and the budget T is small in practice, and we discuss the regimes in which the alternative algorithms win. All code, data and plots are available in the accompanying source package.

Keywords: knapsack problem, dynamic programming, depth-first search, greedy algorithms, pseudo-polynomial time, learning paths, algorithm analysis.

Contents

1 Introduction **1**

1.1 Motivation and problem statement . . . 1
1.2 Techniques and algorithms studied . . . 1
1.3 Contributions . . . 2
1.4 Organization of this report . . . 2

2 Theoretical Background **2**

2.1 Learning-path optimization in e-learning . . . 2
2.2 The 0/1 knapsack problem . . . 3
2.3 Algorithm design paradigms . . . 3
2.3.1 Backtracking and depth-first search . . . 3
2.3.2 Dynamic programming . . . 4
2.3.3 Greedy strategy . . . 4
2.3.4 Branch and Bound . . . 4
2.3.5 Meet-in-the-Middle . . . 4
2.3.6 Approximation schemes . . . 4
2.4 Fundamentals of complexity analysis . . . 5
2.5 Benchmark instance families . . . 5
2.6 From theory to code . . . 5

3 Algorithm Design **5**

3.1 The authors' technique: depth-first search over the decision tree . . . 5
3.2 From the DFS to dynamic programming . . . 6
3.3 The proposed technique: dynamic programming . . . 7
3.4 The comparison technique: greedy strategy . . . 8
3.5 Additional algorithms . . . 8
3.6 The authors' technique on the prerequisite graph . . . 8

4 Implementation **9**

4.1 Environment and architecture . . . 9
4.2 Pseudocode . . . 9
4.3 Test instance . . . 12
4.4 Decisions and missing specifications . . . 13

5 Complexity Analysis **14**

5.1 Conventions . . . 14
5.2 Exhaustive DFS . . . 14

5.3 Dynamic programming . . . 15
5.4 Greedy strategy . . . 16
5.5 Branch and Bound . . . 16
5.6 Meet-in-the-Middle . . . 17
5.7 FPTAS . . . 17
5.8 Summary . . . 17

6 Experimental Evaluation **17**
6.1 Methodology . . . 18
6.2 Correctness validation . . . 18
6.3 Exponential growth of the exhaustive DFS . . . 19
6.4 Dynamic programming and greedy at scale . . . 20
6.5 Pseudo-polynomial behavior . . . 20
6.6 Quality of the greedy heuristic . . . 20
6.7 Exact algorithms across instance families . . . 21
6.8 FPTAS . . . 22
6.9 The article's DFS on a prerequisite graph . . . 23

7 Discussion **23**
7.1 Which algorithm is the most suitable for learning-path selection? . . . 23
7.2 When another algorithm wins . . . 24
7.3 Agreement with the literature . . . 25
7.4 Threats to validity . . . 25

8 Members and Activities **26**

9 Conclusion **26**

References **28**

A Reproducing the Results **29**

1 Introduction

1.1 Motivation and problem statement

Online learning environments offer far more material than any learner can study. A student who can dedicate only a few hours per week must therefore decide *which* learning modules to take, and this decision directly affects how much is learned. Learning-path recommender systems address this decision by personalizing the sequence of learning activities that leads a learner toward a goal [2]. Among the limitations that such systems must respect, the available study time is one of the most concrete: modules take time, and the learner's time is finite.

The article studied in this project, *RUTICO: Recommending Successful Learning Paths Under Time Constraints* [1], addresses exactly this situation. After locating the learner in the course graph, RUTICO uses a depth-first search (DFS) to find all possible paths that fit in the learner's time restriction and recommends the path with the maximum score. The article therefore poses a clear optimization problem (maximize a learning score under a time budget) and presents a clear solution (exhaustive DFS).

We isolate the combinatorial core of this problem. Each learning module $i \in \{1,\dots,n\}$ has an integer study time $w_i \geq 1$ and a non-negative score v_i that quantifies the expected learning benefit; the learner has an integer time budget $T \geq 0$.

Definition 1 (Learning-path selection problem)**.** Given n modules with times w_i and scores v_i and a budget T, find a subset $S \subseteq \{1,\dots,n\}$ that maximizes $v(S) = \sum_{i\in S} v_i$ subject to $w(S) = \sum_{i\in S} w_i \leq T$.

Definition 1 is the classical *0/1 Knapsack Problem*: each module is taken entirely or not at all. In this formulation the modules are independent of each other. The course graph of the article additionally imposes prerequisites; ignoring them is a deliberate *relaxation* that we state explicitly and discuss in Sections 4.4 and 9.

1.2 Techniques and algorithms studied

The assignment requires analyzing the algorithm created by the authors of the article, implementing it, and comparing it with at least one other approach from the literature. We therefore study three algorithms that play distinct roles, and three additional ones that broaden the comparison (Table 1).

Table 1: Roles of the algorithms and design techniques studied.

Role	Algorithm	Design technique	Source file
Authors (article)	Exhaustive DFS with time pruning	Depth-first search with backtracking	`dfs.py`
Proposal	0/1 Knapsack DP (table, vector, memoized)	Dynamic programming	`dp.py`
Comparison (mandatory)	Greedy by score/time ratio	Greedy strategy (and merge sort: divide and conquer)	`greedy.py`
Extra comparison	Branch and Bound	Implicit enumeration with bounding	`branch_and_bound.py`
Extra comparison	Meet-in-the-Middle	Divide and conquer on the input set	`meet_in_the_middle.py`
Extra comparison	FPTAS	Value scaling + DP over the profit	`fptas.py`
Extension	DFS over the prerequisite graph	Graph search with backtracking	`graph_dfs.py`

The central argument of the project is that these techniques are not unrelated. The DFS of the authors explores a decision tree with 2^n leaves in which many branches reach the same state; dynamic programming

is, in essence, the same search with each state computed only once. This explains why DP is the natural proposal: it keeps the exactness of the exhaustive search and removes its exponential redundancy (Section 3).

1.3 Contributions

- A formal model of the learning-path selection problem and its relation to the knapsack family, including the relaxation with respect to the article (Sections 1 and 2).
- A design analysis that derives dynamic programming from the authors' DFS (Section 3).
- Proofs of correctness and of best-, worst- and (empirical) average-case complexity for every algorithm, including approximation guarantees for the greedy variants and the FPTAS (Section 5).
- A modular Python implementation written without algorithmic libraries, in which every algorithm returns a common result type with an operation counter (Section 4).
- An experimental evaluation that validates correctness on more than one thousand random instances, measures growth rates in operations and in time, tests three families of instances with different correlation between time and score, and demonstrates the pseudo-polynomial behavior of DP (Section 6).
- A discussion of why DP is the most suitable algorithm for this scenario, and of the regimes in which each alternative wins (Section 7).

1.4 Organization of this report

Section 2 presents the theoretical background. Section 3 covers the algorithm design (assignment item 1.1.1). Section 4 describes the implementation, pseudocode and the test instance. Section 5 contains the complexity analysis (item 1.1.2). Section 6 reports the experiments, and Section 7 discusses the results. Section 8 lists the activities of each member, and Section 9 concludes the report with the main difficulties encountered.

2 Theoretical Background

This section presents the concepts on which the rest of the report relies. Each subsection ends by indicating the code module in which the concept materializes, so that the theory and the implementation can be read side by side (a summary is given in Table 2).

2.1 Learning-path optimization in e-learning

A learning path is the implementation of a curriculum design: a set of learning activities that helps a user achieve particular learning goals. Because learners differ in their limitations, backgrounds and goals, personalizing these paths became a significant research task, and the survey of Nabizadeh et al. organizes the methods, the parameters used for personalization and the evaluation approaches of this area [2]. Available study time is one of the limitations a learner may have, and it is the one that RUTICO addresses directly [1]: the system looks for paths whose total duration respects the learner's time restriction and whose score is maximal.

When modules have prerequisites, the feasible selections form a structure over a directed acyclic graph (DAG). The corresponding optimization problem is the *precedence-constrained knapsack problem* (PCKP),

in which a module can be adopted only if all its predecessors are in the knapsack [3]. PCKP is NP-hard because it contains the plain knapsack as the special case without precedence, and dynamic programming algorithms for it are known to solve small instances to optimality [3]. Ignoring prerequisites therefore yields a strictly simpler problem whose structure is completely understood, and which is the subject of this work. The graph search of RUTICO itself is represented in our code by `graph_dfs.py`.

2.2 The 0/1 knapsack problem

Definition 1 can be written as the integer linear program

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^{n} w_i x_i \leq T, \quad x_i \in \{0,1\}, \tag{1}$$

where $x_i = 1$ means that module i is selected. Without loss of generality we assume $w_i \leq T$ for every module, since a module that does not fit alone can never be part of a solution.

Computational complexity. The decision version of the problem (is there a subset with $v(S) \geq V$ and $w(S) \leq T$?) is NP-complete; the knapsack problem is among the problems whose NP-completeness was established by Karp [4] and it is a standard example in the theory of NP-completeness [5]. No polynomial-time algorithm is known for it, but it is NP-hard only in the *weak* sense: there are algorithms whose running time is polynomial in the *numeric value* of the input, called pseudo-polynomial algorithms [5, 8]. The input of Definition 1 is encoded with $\Theta\big(n \log(T + \max_i w_i + \max_i v_i)\big)$ bits, so a running time such as $\Theta(nT)$ is exponential in the number of bits used to write T, even though it is linear in the number T itself. This distinction is the key to understanding both the efficiency of dynamic programming in practice and its limitation (Sections 3 and 6). Despite its NP-hardness, many knapsack instances are easy for modern algorithms, and Pisinger studies which classes of instances remain hard [9].

Fractional relaxation. If modules could be taken partially ($x_i \in [0,1]$), the problem would be solved optimally by a greedy rule: sort the modules by decreasing ratio v_i/w_i and take them in this order until the first module that does not fit, of which only a fraction is taken. This observation, due to Dantzig [7], has two consequences that we exploit: it gives an upper bound on the 0/1 optimum (used by Branch and Bound), and it explains both why a ratio-based greedy is attractive and why it fails when items are indivisible.

2.3 Algorithm design paradigms

2.3.1 Backtracking and depth-first search

Backtracking systematically builds candidate solutions and abandons (prunes) a partial candidate as soon as it cannot lead to a feasible solution [15]. It is implemented with a depth-first traversal of the *decision tree*, in which each level corresponds to a decision (here: include or exclude one module) and each leaf to a complete selection. Depth-first search itself is a linear-time graph traversal [16, 18]. A recursive implementation must *restore* the state when it returns from a recursive call (the backtracking step), otherwise decisions of one branch leak into the other. For the knapsack, the tree has 2^n leaves; pruning the "include" branch whenever the time budget would be exceeded removes infeasible subtrees but does not change the exponential worst case. This is the technique of the authors' solution (`dfs.py`, `graph_dfs.py`).

2.3.2 Dynamic programming

Dynamic programming, introduced by Bellman [6], solves an optimization problem by combining solutions of subproblems and is applicable when the problem has two properties [18, 19]:

- **Optimal substructure:** an optimal solution contains optimal solutions of its subproblems (Bellman's principle of optimality).
- **Overlapping subproblems:** the same subproblems are needed repeatedly, so computing each one once and reusing its value saves work.

There are two equivalent implementations. *Bottom-up* fills a table in an order that guarantees that dependencies are ready; *top-down* keeps the recursive formulation and caches results, a technique known as memoization, named after Michie's "memo functions" [17]. Both compute the same values; the top-down version computes only the states that are actually reachable, at the price of recursion overhead. When each row of the table depends only on the previous one, the table can be compressed into a single vector, reducing the space. Dynamic programming is the proposal of this project (`dp.py`: `dp_table`, `dp_vector`, `dp_memoized`).

2.3.3 Greedy strategy

A greedy algorithm builds a solution by repeatedly making the locally best choice. It is correct when the problem has the greedy-choice property and optimal substructure [18]; the fractional knapsack has it (Section 2.2), the 0/1 knapsack does not. Applied to the 0/1 knapsack, the ratio-based greedy is fast but offers no optimality guarantee, and it can be arbitrarily bad (Proposition 8). A simple modification, returning the better of the greedy solution and the best single module, guarantees at least half of the optimum [8] (Proposition 9). The greedy strategy is our mandatory comparison algorithm (`greedy.py`); it relies on a sort, which we implement as a merge sort (divide and conquer, `sorting.py`).

2.3.4 Branch and Bound

Branch and Bound, proposed by Land and Doig for discrete programming [13], explores the decision tree like the DFS but discards a subtree when an *upper bound* on its best possible value does not exceed the best complete solution already found. For the knapsack, the natural bound is the fractional relaxation of Dantzig [7], used in the first branch-and-bound algorithms for the problem [14, 10]. The worst case remains exponential, but the pruning can make the search very fast on instances for which the bound is tight (`branch_and_bound.py`).

2.3.5 Meet-in-the-Middle

Horowitz and Sahni showed how to solve the knapsack problem in $\mathcal{O}(2^{n/2})$ time by splitting the set of items into two halves, enumerating the subsets of each half, and combining them [10]. Its cost does not depend on the numeric value of T, which makes it attractive when T is very large and n is moderate (`meet_in_the_middle.py`).

2.3.6 Approximation schemes

A *fully polynomial-time approximation scheme* (FPTAS) returns, for any $\varepsilon > 0$, a solution of value at least $(1-\varepsilon)\,\mathrm{OPT}$ in time polynomial in both n and $1/\varepsilon$ [21, 22]. The knapsack problem admits an FPTAS,

obtained by scaling down the values and solving the scaled instance exactly with a dynamic program over the *profit* [11, 12]. It trades exactness for independence from T (`fptas.py`).

2.4 Fundamentals of complexity analysis

We use the asymptotic notation $\mathcal{O}$, Ω and Θ [18]. The *best case* and the *worst case* of an algorithm are the minimum and the maximum running time over all inputs of a given size n; the *average case* is the expected running time under an input distribution, which is rarely available in closed form. For this reason we complement the analytical best and worst cases with an *empirical* average case measured on random instances. Two parameters describe the input size of our problem: the number of modules n and the budget T; the algorithms studied depend on n alone, on n and T, or on n and the instance structure.

Recursive algorithms are analyzed with recurrences such as $T(n) = 2T(n/2) + \Theta(n)$, whose solution is given by the master theorem [18]. Following the tradition of Knuth [20], we also analyze algorithms by counting how often their elementary steps are executed. This is the reason why every algorithm in our implementation reports an operation counter: counters are deterministic and independent of the machine, whereas wall-clock time is not. Wall-clock times are reported only as complementary evidence.

2.5 Benchmark instance families

The behavior of knapsack algorithms depends strongly on the relation between times (weights) and scores (profits) [9, 8]. In the standard generators, with R the range of the weights, the *uncorrelated* family draws v_i independently of w_i; the *weakly correlated* family draws v_i near w_i; and the *strongly correlated* family sets $v_i = w_i + c$. Pisinger identifies instance classes that are hard for branch-and-bound algorithms, among them strongly correlated ones, as well as instances with large coefficients that challenge dynamic programming [9]. We adopt three families with these definitions in `instances.py` (Section 6.1). In e-learning, the correlated families are also the plausible ones: our working hypothesis is that longer modules tend to carry more content and therefore a higher score.

2.6 From theory to code

3 Algorithm Design

This section addresses assignment item 1.1.1. For each technique we explain the *motivation*, the *logic* and the *execution steps*, and we show how the techniques relate to one another.

3.1 The authors' technique: depth-first search over the decision tree

Motivation. Without any assumption on the data, the only way to be sure of finding the best selection is to examine every feasible one. RUTICO does so by enumerating the paths of the course graph with a DFS [1]. In the knapsack version of the problem, the enumeration is the traversal of a binary decision tree.

Logic. Each node of the tree is a partial decision (i, u, s): modules $1, \ldots, i$ have been decided, the selected ones use u time units and have accumulated score s. The node has up to two children: *take* module $i+1$ (only if $u + w_{i+1} \leq T$, which is the pruning rule) and *skip* it. Every node corresponds to a feasible selection, so the best score seen over all visited nodes is the optimum.

Table 2: Correspondence between theoretical concepts and code modules.

Concept	Module (function)	Complexity
Decision tree, pruning, backtracking	`dfs.py` (`exhaustive_dfs`)	$\Theta(n)$ to $\Theta(2^n)$
Graph search with backtracking	`graph_dfs.py` (`dfs_paths`)	up to 2^{n-1} paths
Optimal substructure, bottom-up table	`dp.py` (`dp_table`)	$\Theta(nT)$
Space optimization	`dp.py` (`dp_vector`)	$\Theta(nT)$ time, $\Theta(T)$ space
Memoization (top-down)	`dp.py` (`dp_memoized`)	$\leq n(T+1)$ states
Greedy strategy and 1/2-approximation	`greedy.py` (`greedy`, `greedy_with_best_item`)	$\Theta(n \log n)$
Divide and conquer (merge sort)	`sorting.py` (`merge_sort`)	$\Theta(n \log n)$
Branch and Bound with Dantzig bound	`branch_and_bound.py`	$\mathcal{O}(n2^n)$
Meet-in-the-Middle	`meet_in_the_middle.py`	$\mathcal{O}(2^{n/2})$
FPTAS	`fptas.py`	$\mathcal{O}(n^3/\varepsilon)$
Instance families	`instances.py`	—

Execution steps.

1. Start at the root $(0,0,0)$ with an empty list of chosen modules.
2. At every node, if s exceeds the best score so far, store a *copy* of the chosen list.
3. If all modules have been decided ($i = n$), return.
4. If module $i+1$ fits, append it to the chosen list, recurse, and then *remove* it (backtracking). Without this removal the state of one branch would contaminate the other.
5. Recurse on the branch that skips module $i+1$.

The running time is proportional to the number of visited nodes; on the test instance of Section 4.3 the DFS visits 93 nodes.

3.2 From the DFS to dynamic programming

Motivation. The tree of Figure 1 has 2^n leaves, but its nodes are not all different. Consider the node reached after taking module 1 and skipping module 2, and the node reached after skipping module 1 and taking module 2: both have used the same time and therefore have the same remaining time t, and the best completion from them is identical. The DFS computes it twice.

Logic. The key observation is that the best score obtainable from the undecided modules depends only on the pair (i,t) and not on the path that led to it. Let $F(i,t)$ be the best score using modules $i+1,\ldots,n$ within t time units. Then $F(n,t) = 0$ and

$$F(i,t) = \begin{cases} F(i+1,t) & \text{if } w_{i+1} > t, \\ \max\{F(i+1,t),\ v_{i+1} + F(i+1,t-w_{i+1})\} & \text{otherwise.} \end{cases} \tag{2}$$

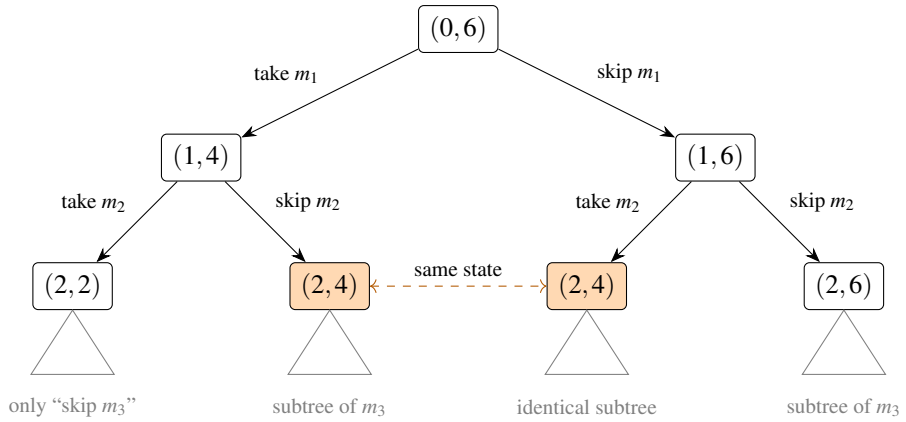


Figure 1: Decision tree for three modules with times $w = (2,2,3)$ and budget $T = 6$. A node (i,t) means that modules $1,\dots,i$ were decided and t time units remain. The two shaded nodes are the same state, reached by different decisions, and their subtrees are identical: the DFS explores both, a dynamic program explores one.

There are only $n(T+1)$ distinct pairs (i,t) with $0 \le i < n$, in contrast to the 2^n leaves. Caching F in the recursion gives the *memoized* top-down algorithm (`dp_memoized`); filling a table in dependency order gives the *bottom-up* algorithm (`dp_table`). The table in the code uses the equivalent prefix formulation of Section 3.3. In short, **dynamic programming is the authors' DFS with memoization**, which is why it keeps the exactness of the exhaustive search and discards its redundancy.

3.3 The proposed technique: dynamic programming

Logic. Let $D[i][t]$ be the best score obtainable from the first i modules within at most t time units. Then $D[0][t] = 0$ for every t, and for $i \ge 1$

$$D[i][t] = \begin{cases} D[i-1][t] & \text{if } w_i > t, \\ \max\{D[i-1][t],\ D[i-1][t-w_i] + v_i\} & \text{otherwise.} \end{cases} \tag{3}$$

The first case says that module i does not fit and cannot be used. In the second case there are exactly two options: leave module i out (the best is $D[i-1][t]$) or take it, which consumes w_i units and adds v_i points, leaving $t - w_i$ units for the first $i-1$ modules. The recurrence refers to row $i-1$, never to row i, so each module is used at most once: this is what makes the problem 0/1 instead of unbounded.

Execution steps.

1. *Initialization:* fill row 0 with zeros.
2. *Fill:* for $i = 1,\dots,n$ and $t = 0,\dots,T$, apply Equation (3).
3. *Read the optimum:* $D[n][T]$ is the maximum score.
4. *Reconstruct:* start at $(i,t) = (n,T)$. If $D[i][t] \ne D[i-1][t]$, module i was taken: record it and set $t \leftarrow t - w_i$. In either case set $i \leftarrow i-1$.

Useful properties. Because $D[i][t]$ is defined for *at most* t units, the last row $D[n][\cdot]$ contains the optimum for *every* budget up to T, and the reconstruction works for any of them. This answers for free the natural question of a learner: "what if I had only 8 hours?". The table needs $(n+1)(T+1)$ cells; since a row depends only on the previous one, a single vector of $T+1$ cells suffices if t is scanned in decreasing order

(otherwise the already-updated value would be reused and a module could be taken several times) at the price of losing the reconstruction (`dp_vector`).

3.4 The comparison technique: greedy strategy

Motivation. The fractional knapsack is solved exactly by taking modules in decreasing order of v_i/w_i (Section 2.2). It is natural to try the same rule when modules are indivisible: it is simple and extremely fast.

Logic and steps. (1) Sort the modules by decreasing ratio; (2) scan them in this order and take every module that still fits. The sort is a merge sort, an instance of divide and conquer: *divide* the list into two halves, *conquer* by sorting each half recursively, *combine* by merging the two sorted halves. Ratios are compared by cross-multiplication ($v_i w_j > v_j w_i$), avoiding divisions and floating-point issues, and the merge is stable.

Why it can fail. Because modules cannot be split, a short module with a high ratio can consume time that prevents a better combination. On the test instance the greedy takes modules 1, 2, 4 and 6 (9 h, score 77) while the optimum is modules 2 and 3 (10 h, score 91). The variant `greedy_with_best_item` returns the better of the greedy solution and the best single module, which repairs the worst pathologies (Proposition 9).

3.5 Additional algorithms

Branch and Bound. *Motivation:* the DFS explores subtrees that clearly cannot win. *Logic:* sort the modules by ratio; at each node compute an upper bound by filling the remaining room with whole modules in ratio order and a fraction of the first one that does not fit (floored, because scores are integers); if the bound does not exceed the best known score, discard the subtree. *Steps:* as in the DFS, plus the bound test before expanding a node.

Meet-in-the-Middle. *Motivation:* reduce 2^n to about $2^{n/2}$ without depending on T. *Logic:* split the modules into halves A and B; enumerate the feasible subsets of each half sorted by time; for each subset of A, the best partner in B is the best subset whose time is at most T minus the time of the A-subset. *Steps:* (1) generate each half's subsets by merging a list with its shifted copy, which keeps it sorted by time; (2) precompute for B the best score among the subsets up to each position; (3) sweep A in increasing time while a pointer in B moves monotonically backwards.

FPTAS. *Motivation:* obtain a guarantee that does not depend on T. *Logic:* scale the scores to $v'_i = \lfloor v_i/K \rfloor$ with $K = \varepsilon v_{\max}/n$, which makes the sum of scaled scores at most n^2/ε, and solve the scaled instance exactly with a DP over the *profit*: for each scaled score, the minimum time needed to reach it. *Steps:* fill the profit table, pick the largest scaled score reachable within T, reconstruct the modules and sum their *original* scores.

3.6 The authors' technique on the prerequisite graph

The extension `graph_dfs.py` implements the strategy of the article on a graph of modules with prerequisites: starting from a source module, it enumerates every simple path whose accumulated time

fits the budget, records the path with the highest score and uses backtracking to leave each path. On a complete DAG (an edge from i to every $j > i$) every subset of the other $n-1$ nodes, taken in order, is a path, so there are 2^{n-1} paths; Section 6.9 confirms this count.

4 Implementation

4.1 Environment and architecture

The prototype is written in Python (the reported experiments ran on Python 3.12; version 3.10 or newer is required) using only the standard library; the matplotlib library is used exclusively to draw the plots of Section 6 and does not take part in any algorithm. Even the sorting routine is hand-written. The code runs on Linux with a single command (Appendix A). Its organization follows the two files requested by the assignment: the *Algorithm* files and the *Experiment* file.

Table 3: Organization of the source code.

File	Content
`algorithms/model.py`	`LearningModule` (name, time, score), `Result` (algorithm, value, indices, operations), instance validation and evaluation of a selection
`algorithms/sorting.py`	Stable hand-written merge sort; `sort_by_ratio` compares ratios by cross-multiplication
`algorithms/dfs.py`	`exhaustive_dfs`: the authors' strategy (decision tree, pruning, backtracking)
`algorithms/graph_dfs.py`	`dfs_paths`, `complete_dag`: DFS over the prerequisite graph
`algorithms/dp.py`	`dp_table`, `dp_vector`, `dp_memoized`: the proposed dynamic programming
`algorithms/greedy.py`	`greedy`, `greedy_with_best_item`: comparison algorithm and its variant with a guarantee
`algorithms/branch_and_bound.py`	`branch_and_bound`
`algorithms/meet_in_the_middle.py`	`meet_in_the_middle`
`algorithms/fptas.py`	`fptas`
`instances.py`	`generate_instance`: the three instance families, reproducible by seed
`experiment.py`	**Experiment file**: the test instance and nine blocks of experiments (command-line selectable)

Every algorithm returns the same `Result` type, which makes comparisons and tests uniform and avoids duplicated code. The field `operations` is a deterministic counter of elementary steps whose meaning is fixed per algorithm (Table 4).

4.2 Pseudocode

Algorithm 1: Exhaustive DFS with pruning and backtracking (`exhaustive_dfs`)

Input: modules (w_i, v_i), $i = 1..n$; budget T
Output: best score *best* and the selected modules *bestSet*

best $\leftarrow 0$; *chosen* $\leftarrow [\,]$; *bestSet* $\leftarrow [\,]$;
Procedure Explore(i, u, s)
  if $s >$ *best* **then**
    best $\leftarrow s$; *bestSet* $\leftarrow$ copy(*chosen*);
  if $i = n$ **then return**;
  if $u + w_{i+1} \le T$ **then**
    append $i+1$ to *chosen*;
    Explore$(i+1,\ u+w_{i+1},\ s+v_{i+1})$;
    remove $i+1$ from *chosen* ; ▷ *backtracking*
  Explore$(i+1,\ u,\ s)$;
Explore$(0,0,0)$;
return *best*, *bestSet*;

Algorithm 2: Dynamic programming: table and reconstruction (`dp_table`)

Input: modules (w_i, v_i), $i = 1..n$; budget T
Output: optimal score $D[n][T]$ and the selected modules *chosen*

for $t \leftarrow 0$ **to** T **do** $D[0][t] \leftarrow 0$;
for $i \leftarrow 1$ **to** n **do**
  for $t \leftarrow 0$ **to** T **do**
    $D[i][t] \leftarrow D[i-1][t]$; ▷ *do not take module i*
    if $w_i \le t$ ***and*** $D[i-1][t-w_i] + v_i > D[i][t]$ **then**
      $D[i][t] \leftarrow D[i-1][t-w_i] + v_i$; ▷ *take module i*
$t \leftarrow T$; *chosen* $\leftarrow [\,]$;
for $i \leftarrow n$ ***down to*** 1 **do**
  if $D[i][t] \neq D[i-1][t]$ **then**
    prepend i to *chosen*; $t \leftarrow t - w_i$;
return $D[n][T]$, *chosen*;

Algorithm 3: Dynamic programming with memoization (`dp_memoized`)

Input: modules (w_i, v_i), $i = 1..n$; budget T
Output: optimal score Best$(0, T)$

memo $\leftarrow$ empty map;
Procedure Best(i,t)
  if $i = n$ **then return** 0;
  if $(i,t) \in$ *memo* **then return** *memo*$[(i,t)]$;
  $r \leftarrow$ Best$(i+1,t)$;
  if $w_{i+1} \le t$ **then** $r \leftarrow \max\{r,\ v_{i+1} + \text{Best}(i+1, t-w_{i+1})\}$;
  memo$[(i,t)] \leftarrow r$; **return** r;

Algorithm 4: Greedy by ratio, with the best-single-module variant (`greedy`, `greedy_with_best_item`)

Input: modules (w_i, v_i), $i = 1..n$; budget T
Output: score G (greedy) and score $\max\{G, M\}$ (variant)

$order \leftarrow$ MERGESORT(modules, i before j iff $v_i w_j > v_j w_i$);
$u \leftarrow 0$; $G \leftarrow 0$;
foreach *i in order* **do**
  if $u + w_i \le T$ **then**
    $u \leftarrow u + w_i$; $G \leftarrow G + v_i$;
$M \leftarrow \max\{v_i : w_i \le T\}$; ▷ *variant only*
return G **and** $\max\{G, M\}$;

Algorithm 5: Branch and Bound with the fractional bound (`branch_and_bound`)

Input: modules (w_i, v_i), $i = 1..n$; budget T
Output: optimal score *best*

$o \leftarrow$ modules sorted by decreasing v/w; $best \leftarrow 0$; $chosen \leftarrow [\,]$;
Procedure UpperBound(k, u, s)
  $room \leftarrow T - u$; $ub \leftarrow s$;
  for $j \leftarrow k+1$ **to** n **do**
    if $w_{o_j} \le room$ **then** $room \leftarrow room - w_{o_j}$; $ub \leftarrow ub + v_{o_j}$;
    else return $ub + \lfloor room \cdot v_{o_j} / w_{o_j} \rfloor$;
  return ub;
Procedure Explore(k, u, s)
  $best \leftarrow \max\{best, s\}$;
  if $k = n$ ***or*** UpperBound$(k, u, s) \le best$ **then return**;
  if $u + w_{o_{k+1}} \le T$ **then**
    take o_{k+1}, Explore$(k+1,\ u + w_{o_{k+1}},\ s + v_{o_{k+1}})$, undo;
  Explore$(k+1,\ u,\ s)$;
Explore$(0, 0, 0)$;

Algorithm 6: Meet-in-the-Middle (`meet_in_the_middle`)

Input: modules (w_i, v_i), $i = 1..n$; budget T
Output: optimal score *best*

split the modules into halves A (first $\lfloor n/2 \rfloor$) and B (the rest);
Procedure Subsets(H)
  $L \leftarrow [(0, 0)]$; ▷ *pairs (time, score), sorted by time*
  foreach *module j in H* **do**
    $L' \leftarrow [(t + w_j,\ s + v_j) : (t, s) \in L,\ t + w_j \le T]$;
    $L \leftarrow$ MERGE(L, L') by time ; ▷ *stays sorted*
  return L;
$L_A \leftarrow$ Subsets(A); $L_B \leftarrow$ Subsets(B);
$bestB[j] \leftarrow$ largest score among $L_B[0..j]$; $p \leftarrow |L_B| - 1$; $best \leftarrow 0$;
foreach (t, s) ***in*** L_A (increasing t) **do**
  while *time of* $L_B[p] + t > T$ **do**
    $p \leftarrow p - 1$;
  $best \leftarrow \max\{best,\ s + bestB[p]\}$;
return *best*;

Table 4: What each operation counter counts.

Algorithm	Counted operations
`exhaustive_dfs`	nodes of the decision tree visited
`dp_table`	table cells computed, exactly $n(T+1)$
`dp_vector`	iterations of the inner loop
`dp_memoized`	distinct states (i,t) computed ($\leq n(T+1)$)
`greedy`	comparisons of the merge sort plus one step per module scanned
`greedy_with_best_item`	as `greedy`, plus n steps to find the best single module
`branch_and_bound`	nodes visited, plus steps spent computing upper bounds, plus sort comparisons
`meet_in_the_middle`	subset entries generated, merge comparisons and pointer steps
`fptas`	cells of the profit table

Algorithm 7: FPTAS by profit scaling (`fptas`)

Input: modules (w_i, v_i) with $w_i \leq T$; budget T; accuracy ε
Output: a selection with score $\geq (1-\varepsilon)\,\mathrm{OPT}$

$K \leftarrow \varepsilon \cdot v_{\max}/n$; $\quad v'_i \leftarrow \lfloor v_i/K \rfloor$; $\quad S \leftarrow \sum_i v'_i$;
$W[0][0] \leftarrow 0$; $\quad W[0][s] \leftarrow \infty$ for $s > 0$; $\quad \triangleright$ $W[k][s]$*: least time to reach scaled score* s
for $k \leftarrow 1$ **to** n **do**
  for $s \leftarrow 0$ **to** S **do**
    $W[k][s] \leftarrow W[k-1][s]$;
    if $s \geq v'_k$ ***and*** $W[k-1][s-v'_k] + w_k \leq T$ ***and*** $W[k-1][s-v'_k] + w_k < W[k][s]$ **then**
      $W[k][s] \leftarrow W[k-1][s-v'_k] + w_k$;
$s^* \leftarrow \max\{s : W[n][s] \leq T\}$; $\quad$ reconstruct the modules as in Algorithm 2;
return the selected modules and the sum of their *original* scores;

4.3 Test instance

The test instance models a learner with a budget of $T = 10$ hours choosing among six modules (Table 5). It is defined in `test_instance()` and executed by the block `demo` of `experiment.py`.

Table 5: Test instance ($T = 10$ h).

i	Module	Hours w_i	Score v_i	Ratio v_i/w_i
1	Programming Logic	1	10	10.0
2	Python Basics	5	46	9.2
3	Data Structures	5	45	9.0
4	Databases	2	15	7.5
5	Networking	3	20	6.7
6	Git	1	6	6.0

Table 6 shows the table $D[i][t]$ produced by the code. The shaded cells are visited by the reconstruction, which starts at $D[6][10] = 91$. At $i = 6$, 5 and 4 the value does not change when module i is removed ($D[i][10] = D[i-1][10]$), so Git, Networking and Databases are not taken. At $i = 3$, $D[3][10] = 91 \neq D[2][10] = 56$: Data Structures is taken and the remaining time is $10 - 5 = 5$. At $i = 2$, $D[2][5] = 46 \neq D[1][5] = 10$: Python Basics is taken and the remaining time is 0. The optimal selection is therefore modules 2 and 3, using 10 hours and scoring 91.

The greedy algorithm sorts the modules by ratio (1, 2, 3, 4, 5, 6), takes modules 1, 2, 4 and 6 (9 hours, score 77), and skips 3 and 5 because they no longer fit. It ends 15.4% below the optimum because module

Table 6: Dynamic-programming table for the test instance. Rows: modules considered; columns: available time t. Shaded: cells visited by the reconstruction; bold: modules taken.

$i \backslash t$	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10	10	10	10	10
2	0	10	10	10	10	**46**	56	56	56	56	56
3	0	10	10	10	10	46	56	56	56	56	**91**
4	0	10	15	25	25	46	56	61	71	71	91
5	0	10	15	25	30	46	56	61	71	76	91
6	0	10	16	25	31	46	56	62	71	77	91

1, with the highest ratio, uses an hour that prevents the ideal pair from fitting. Table 7 lists the result of every algorithm; all exact algorithms return 91.

Table 7: Results on the test instance (output of `python experiment.py -only demo`).

Algorithm	Score	Selected modules	Hours used	Operations
Exhaustive DFS	91	2, 3	10	93
DP (table)	91	2, 3	10	66 ($= 6 \times 11$)
DP (vector)	91	— (value only)	—	49
DP (memoized)	91	2, 3	10	32 states
Branch and Bound	91	2, 3	10	70
Meet-in-the-Middle	91	2, 3	10	46
FPTAS ($\varepsilon = 0.1$)	91	2, 3	10	1,098
Greedy	77	1, 2, 4, 6	9	13
Greedy + best item	77	1, 2, 4, 6	9	19

The memoized DP visits 32 states, well below the 66 cells of the table and the 93 nodes of the DFS, which illustrates the link between the three algorithms discussed in Section 3.2. An *adversarial instance* shows that the greedy can be arbitrarily bad: with $T = 1000$, a short module with 1 hour and 2 points (ratio 2.0) and a long module with 1000 hours and 1000 points (ratio 1.0). The greedy takes the short module and obtains 2 (a gap of 99.8%), after which the long module no longer fits; the variant with the best single module, the DP and all exact algorithms obtain 1000.

4.4 Decisions and missing specifications

The assignment asks for the decisions made and for any detail missing from its description. Ours were:

1. **Relaxation without prerequisites.** The article works on a course graph; we treat modules as independent (0/1 knapsack). The version with precedence (Section 2) is left as future work. The prerequisite-graph DFS is provided separately in `graph_dfs.py`.
2. **Time and score are inputs.** We do not reimplement how the article estimates module time and score; we take them as given integers.
3. **Integer time unit.** Times are positive integers (minutes or blocks of 15 minutes). This is required by the DP table and is the origin of its pseudo-polynomial nature.
4. **Details a DFS implementation must fix.** State restoration is explicit (the `pop` after the recursive

call); the best selection is recorded at every node; ties keep the first optimum found (strict comparison); modules are visited in input order (only Branch and Bound uses ratio order).

5. **Own merge sort.** We implement a stable merge sort instead of the built-in `sorted`. This keeps the greedy at $\Theta(n \log n)$ in every case (Python's Timsort runs in $\mathcal{O}(n)$ on already sorted input, which would blur the best-case analysis) and respects the rule against relying on library implementations.
6. **Exact arithmetic.** Ratios are compared by cross-multiplication and scores are integers, so there are no floating-point ties in the exact algorithms; only the FPTAS uses a floating-point scaling constant.
7. **Counters and timing.** Conclusions rely on operation counters (Table 4); wall-clock time is the median of three executions (one execution when the DFS exceeds 18 modules).
8. **DFS size limit.** The DFS is run only up to $n = 22$ in the timing experiments, because its cost doubles with each additional module.
9. **Instance families.** The strongly correlated family uses the constant $R/2$ instead of the $R/10$ usual in the literature, to make the effect clearly visible with small ranges; this is a deviation we state explicitly (Section 6.1).

5 Complexity Analysis

This section addresses assignment item 1.1.2. For each algorithm we prove its correctness (when the analysis depends on it) and derive its best-case, worst-case and space complexity; the average case is measured empirically in Section 6, because it depends on the distribution of the instances.

5.1 Conventions

An *elementary operation* is the unit counted by the corresponding counter in Table 4. We use n for the number of modules, T for the budget and $w_i \geq 1$ for integer times. Inputs in which some or all modules do not fit are valid and are accepted by the code (the best case of the DFS occurs on one of them). When we reason about solutions (approximation guarantees), we assume without loss of generality that $w_i \leq T$ for every i, since a module that does not fit alone belongs to no feasible solution and removing it changes neither OPT nor the value returned by the algorithms.

5.2 Exhaustive DFS

Theorem 1 (Correctness of the DFS)**.** *Algorithm 1 returns the maximum score over all feasible selections.*

Proof. Every node (i, u, s) visited corresponds to the feasible selection *chosen* $\subseteq \{1, \ldots, i\}$ with time $u \leq T$ and score s, because a module is appended only if $u + w_{i+1} \leq T$ and is removed on backtracking. Conversely, every feasible selection S is visited: following the decisions of S from the root, every "take" decision is allowed because the accumulated time never exceeds $w(S) \leq T$. The algorithm records the best score over all visited nodes, hence it returns $\max\{v(S) : w(S) \leq T\}$. □

Proposition 2 (Number of nodes)**.** *Let N be the number of nodes visited by Algorithm 1. Then $n+1 \leq N \leq 2^{n+1} - 1$. Moreover, $N = 2^{n+1} - 1$ if and only if $\sum_i w_i \leq T$, and $N = n + 1$ if and only if $w_i > T$ for all i.*

Proof. The "skip" branch is always available, so the path that skips every module exists and has $n+1$ nodes: $N \geq n+1$. Each internal node has at most two children and the depth is n, so the tree has at most $\sum_{k=0}^{n} 2^k = 2^{n+1} - 1$ nodes. Equality holds exactly when every "take" branch is feasible, in which case N satisfies $N(i) = 1 + 2N(i+1)$ with $N(n) = 1$, i.e. $N = 2^{n+1} - 1$. This happens when the path that takes all modules is feasible, which is equivalent to $\sum_i w_i \leq T$ (all other paths use less time). The lower bound is attained when no "take" branch is ever feasible; as the all-skip path keeps the used time at zero, this means $w_i > T$ for all i. □

Best, worst and space. The running time is $\Theta(N)$ (up to the cost of copying the incumbent solution, $\mathcal{O}(n)$ each time it improves; this is the usual convention for tree searches and the factor can be removed by recording the incumbent differently). Hence the best case is $\Theta(n)$ and the worst case is $\Theta(2^n)$. The recursion depth is $n+1$ and the list *chosen* has at most n elements, so the space is $\Theta(n)$. The average case depends on how generous the budget is; it is measured in Section 6.3.

5.3 Dynamic programming

Lemma 3 (Recurrence)**.** *For all $0 \leq i \leq n$ and $0 \leq t \leq T$, the value $D[i][t]$ computed by Equation (3) equals* $\max\{v(S) : S \subseteq \{1,\ldots,i\},\ w(S) \leq t\}$.

Proof. By induction on i. For $i = 0$ the only subset is $\emptyset$, whose score is 0, as in $D[0][t] = 0$. Let $i \geq 1$ and let S be an optimal subset for (i,t). If $w_i > t$, module i cannot belong to S, so S is a feasible subset of $\{1,\ldots,i-1\}$ and, by induction, its value is at most $D[i-1][t]$, which is achievable; this is the first case of the recurrence. If $w_i \leq t$, either $i \notin S$, and then $v(S) \leq D[i-1][t]$; or $i \in S$, and then $S \setminus \{i\}$ is a subset of $\{1,\ldots,i-1\}$ with time at most $t - w_i$, so $v(S) \leq v_i + D[i-1][t-w_i]$. Both values are achievable by the induction hypothesis, hence the optimum is their maximum, as in the second case. □

Lemma 4 (Reconstruction)**.** *The backward walk of Algorithm 2 returns a feasible selection with score $D[n][T]$.*

Proof. Invariant: at the start of the iteration for i with current time t, the selected modules so far plus the optimal value of (i,t) add up to $D[n][T]$. If $D[i][t] \neq D[i-1][t]$, then by the recurrence $D[i][t] > D[i-1][t]$ and $D[i][t] = D[i-1][t-w_i] + v_i$: module i is taken, the invariant holds for $(i-1, t-w_i)$, and the time remains non-negative because $w_i \leq t$. If $D[i][t] = D[i-1][t]$, the invariant holds for $(i-1,t)$ without taking i. At $i = 0$ the remaining value is 0. □

Theorem 5 (Complexity of the DP)**.** *Algorithm 2 computes exactly $n(T+1)$ cells, each in constant time. Its running time is $\Theta(nT)$ in the best, worst and average cases, and the reconstruction adds $\Theta(n)$. The space is $\Theta(nT)$ for the table and $\Theta(T)$ for the vector version (which runs at most $n(T+1)$ inner iterations).*

Proof. The loops have fixed bounds n and $T+1$ that do not depend on the values of w_i and v_i, and the body does a constant number of comparisons and additions. Hence the count is $n(T+1)$ on every input. The table has $(n+1)(T+1)$ cells. In the vector version, row i is computed from row $i-1$ in place by scanning t from T down to w_i; the decreasing order guarantees that $d[t-w_i]$ still holds the value of row $i-1$. □

Proposition 6 (Pseudo-polynomial time)**.** *The running time $\Theta(nT)$ is polynomial in the numeric value of T but exponential in the length of its binary encoding.*

Proof. Writing T takes $k = \lceil \log_2(T+1) \rceil$ bits, so $T \geq 2^{k-1}$ and the running time is $\Omega(n2^{k-1})$, exponential in k. It is polynomial only if T is bounded by a polynomial in n (or if numbers are encoded in unary). This does not contradict the NP-hardness of the problem, which would be contradicted only by an algorithm polynomial in the *bit length* of the input. □

Memoized version. The reachable states (i,t) satisfy $0 \leq i < n$ and $0 \leq t \leq T$, so at most $n(T+1)$ states are computed, each in constant time besides the recursive calls: $\mathcal{O}(nT)$ in the worst case. If no module fits, only the states $(0,T),\ldots,(n-1,T)$ are visited: the best case is $\Theta(n)$. The space is $\mathcal{O}(nT)$ for the cache and $\Theta(n)$ for the recursion.

5.4 Greedy strategy

Proposition 7 (Running time)**.** *The greedy algorithm runs in* $\Theta(n \log n)$ *in the best, worst and average cases, using* $\Theta(n)$ *space.*

Proof. Merging two sorted lists of sizes $a \leq b$ takes between a and $a+b-1$ comparisons. For the balanced halves of merge sort, the cost $m(n)$ of the merge step satisfies $\lfloor n/2 \rfloor \leq m(n) \leq n-1$, so the number of comparisons $C(n) = C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + m(n)$ is $\Theta(n \log n)$ by the master theorem, regardless of the order of the input. The scan that follows takes $\Theta(n)$ steps. The lists and the output use $\Theta(n)$ space. □

Proposition 8 (The greedy has no constant approximation ratio)**.** *For every* $T \geq 3$ *there is an instance on which the greedy obtains value* 2 *while* $\mathrm{OPT} = T$*; the ratio* $2/T$ *tends to* 0*.*

Proof. Take two modules $(w_1, v_1) = (1,2)$ and $(w_2, v_2) = (T,T)$. The ratios are 2 and 1, so the greedy takes module 1 first; then $1+T > T$ and module 2 does not fit. The greedy value is 2, while module 2 alone is feasible and has value $T > 2$. □

Proposition 9 (Greedy with the best single module)**.** *Let* G *be the value of the greedy solution and* $M = \max\{v_i : w_i \leq T\}$*. Then* $\max\{G, M\} \geq \mathrm{OPT}/2$*.*

Proof. Order the modules by decreasing ratio. If all modules fit together, $G = \mathrm{OPT}$. Otherwise let k be the first module in this order that does not fit when the greedy reaches it; the greedy has taken all modules $1,\ldots,k-1$, so $G \geq P := \sum_{i<k} v_i$. By Dantzig's fractional relaxation [7], $\mathrm{OPT} \leq P + \frac{r}{w_k} v_k \leq P + v_k$, where $r < w_k$ is the room left after the first $k-1$ modules. Since $w_k \leq T$, we have $v_k \leq M$, so $\mathrm{OPT} \leq G + M \leq 2\max\{G, M\}$. □

5.5 Branch and Bound

Lemma 10 (Validity of the bound)**.** *For a node at position k with used time u and score s, the quantity* $s + \lfloor \mathrm{fill} \rfloor$ *computed by* UPPERBOUND *is at least the best score of any selection that extends the node.*

Proof. The undecided modules, taken in decreasing ratio, are filled until the first one that does not fit, of which a fraction is taken: this is the optimal solution of the fractional relaxation of the remaining problem [7]. Any integral extension is feasible for the relaxation, so its value is at most the relaxation value; since scores are integers, it is also at most the floor of that value. □

Correctness and complexity. Pruning a node whose bound does not exceed the incumbent discards only extensions that cannot improve it (Lemma 10), so Branch and Bound returns the optimum, by the same argument as Theorem 1. It visits at most $2^{n+1}-1$ nodes and spends $\mathcal{O}(n)$ steps on the bound of each, so the worst case is $\mathcal{O}(n2^n)$. The sort costs $\Theta(n\log n)$, which is also the best case (for example, when no module fits). The behavior in between depends on how tight the bound is, which in turn depends on the instance family (Section 6.7). The space is $\Theta(n)$.

5.6 Meet-in-the-Middle

Theorem 11 (Meet-in-the-Middle)**.** *The algorithm is exact and runs in $\mathcal{O}(2^{\lceil n/2\rceil})$ time and space, regardless of T. This bound is tight (worst case $\Theta(2^{n/2})$) when $\sum_i w_i \le T$, and the best case is $\Theta(n)$ when no module fits.*

Proof. After processing k modules, each list holds at most 2^k subsets, and the merge of a list with its shifted copy costs $\mathcal{O}(2^k)$. Summing over the $h=\lceil n/2\rceil$ modules of the larger half gives $\sum_{k<h}\mathcal{O}(2^k)=\mathcal{O}(2^h)$; the sweep of the second phase is linear in $|L_A|+|L_B|$. Correctness: as t (the time of the subset of A) grows, the room $T-t$ shrinks, so the pointer in L_B only moves backwards; $bestB[p]$ is the best score among the subsets of B with time at most the time at position p, hence the sweep evaluates $\max_a\left(v_a+\text{best}_B(T-t_a)\right)$ over all subsets a of A, which equals OPT. If $\sum_i w_i \le T$ nothing is pruned and both lists are complete. If no module fits, both lists remain $[(0,0)]$ and the merges are trivial: $\Theta(n)$. □

5.7 FPTAS

Theorem 12 (FPTAS guarantee and cost)**.** *For $\varepsilon\in(0,1)$, Algorithm 7 returns a feasible selection with value at least $(1-\varepsilon)$OPT in $\mathcal{O}(n^3/\varepsilon)$ time and space.*

Proof. Let S^* be an optimal selection and S' the selection returned, optimal for the scaled scores $v'_i=\lfloor v_i/K\rfloor$ (the profit DP is exact). Since $Kv'_i\le v_i<Kv'_i+K$,

$$v(S') \;\ge\; K\,v'(S') \;\ge\; K\,v'(S^*) \;>\; \sum_{i\in S^*}(v_i-K) \;\ge\; \text{OPT}-nK \;=\; \text{OPT}-\varepsilon v_{\max} \;\ge\; (1-\varepsilon)\text{OPT},$$

where the last step uses $v_{\max}\le\text{OPT}$ (the module with the largest score fits alone). The scaled scores satisfy $v'_i\le v_{\max}/K=n/\varepsilon$, so $S=\sum_i v'_i\le n^2/\varepsilon$ and the table has $(n+1)(S+1)=\mathcal{O}(n^3/\varepsilon)$ cells, each computed in constant time. □

5.8 Summary

The table summarizes the central trade-off. The algorithms that are exact and independent of the data (DP) pay a cost proportional to the *magnitude* of T; the exact algorithms that depend on n alone (DFS, Branch and Bound, Meet-in-the-Middle) pay an exponential cost in n in the worst case; the greedy is the cheapest but carries no guarantee (Proposition 8); and the FPTAS buys independence from T at the price of exactness and a high polynomial cost.

6 Experimental Evaluation

The experiments confirm the analysis of Section 5 and compare the algorithms on instances of different structure. All of them are implemented in `experiment.py` and can be reproduced with the commands

Table 8: Complexity summary. Averages marked "measured" are reported in Section 6.

Algorithm	Best case	Worst case	Average case	Space
Exhaustive DFS	$\Theta(n)$	$\Theta(2^n)$	measured (data dependent)	$\Theta(n)$
DP (table)	$\Theta(nT)$	$\Theta(nT)$	$\Theta(nT)$	$\Theta(nT)$
DP (vector)	$\Theta(nT)$	$\Theta(nT)$	$\Theta(nT)$	$\Theta(T)$
DP (memoized)	$\Theta(n)$ states	$\mathcal{O}(nT)$ states	measured (reachable states)	$\mathcal{O}(nT)$
Greedy	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Greedy + best item	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Branch and Bound	$\Theta(n \log n)$	$\mathcal{O}(n2^n)$	measured (instance dependent)	$\Theta(n)$
Meet-in-the-Middle	$\Theta(n)$	$\Theta(2^{n/2})$	measured (depends on T)	$\mathcal{O}(2^{n/2})$
FPTAS	$\Theta(nS)$	$\mathcal{O}(n^3/\varepsilon)$	$\Theta(nS)$ with $S \le n^2/\varepsilon$	$\Theta(nS)$

of Appendix A; every table of this section is generated from the CSV files that the code writes to `results/`.

6.1 Methodology

Environment and measures. The experiments ran on a single Linux machine with Python 3.12. We report two measures. *Operation counters* (Table 4) are deterministic and independent of the hardware, and are the basis of the conclusions. *Wall-clock time* is measured with `time.perf_counter` as the median of three executions (a single execution for the DFS beyond 18 modules) and is only complementary evidence, since it varies from machine to machine. The quality of a heuristic is measured by the *gap* $(\text{OPT} - \text{value})/\text{OPT}$, where OPT is computed by the DP.

Instance families. Let R be the maximum module time. In all families the times w_i are drawn uniformly from $\{1,\dots,R\}$. The *uncorrelated* family draws v_i uniformly from $\{1,\dots,R\}$ independently of w_i. The *weakly correlated* family sets $v_i = \max\{1, w_i + \delta_i\}$ with δ_i uniform in $[-\lfloor R/10 \rfloor, \lfloor R/10 \rfloor]$. The *strongly correlated* family sets $v_i = w_i + \lfloor R/2 \rfloor$. As noted in Section 4.4, the literature usually adds $R/10$ in the strongly correlated case; we use a larger constant so that the effect is evident with small ranges. The budget is $T = \max\{1, \lfloor \rho \sum_i w_i \rfloor\}$ for a *budget ratio* ρ. Unless stated otherwise $R = 20$. All instances are generated from fixed seeds, so every experiment is reproducible.

6.2 Correctness validation

Before any measurement, the implementations were validated by five edge cases (no modules, $T = 0$, no module fits, all modules fit, and tied scores) and by 1,200 random instances with $n \le 16$, drawn from the three families with $\rho \in \{0.1, 0.3, 0.5, 0.7, 1.0\}$ and $R \in \{5, 20\}$. For each instance, the exhaustive DFS is the reference. The checks were: (i) all exact algorithms (DFS, the three DP versions, Branch and Bound, Meet-in-the-Middle) return the same optimal value; (ii) every returned selection is feasible and its score equals the reported value; (iii) the greedy value never exceeds the optimum; (iv) the greedy with the best single module reaches at least half of the optimum (Proposition 9); (v) the FPTAS with $\varepsilon = 0.1$ stays between 90% and 100% of the optimum (Theorem 12). **Result: 0 failures.** This also validates the comparison baseline used in the following experiments.

6.3 Exponential growth of the exhaustive DFS

Table 9 compares the exact algorithms for n from 10 to 22. In the worst case for the DFS (all modules fit), the number of nodes is *exactly* $2^{n+1}-1$, as stated by Proposition 2: 2,047 for $n=10$ and 8,388,607 for $n=22$, with wall-clock time growing from 0.0003 s to 1.24 s. Each extra module doubles the cost (Figure 2).

Table 9: Exact algorithms for small n (uncorrelated family, $R=20$). Operation counts; the average case uses five instances per size.

	Worst case ($T=\sum w_i$)			Average case ($T=50\%$ of $\sum w_i$, 5 instances)			
n	DFS nodes	$2^{n+1}-1$	DFS time (s)	DFS nodes	B&B ops	MITM ops	DP ops
10	2,047	2,047	0.0003	1,229	108	222	496
12	8,191	8,191	0.0012	4,985	126	466	850
14	32,767	32,767	0.0046	19,146	191	962	930
16	131,071	131,071	0.0238	75,602	192	1,961	1,395
18	524,287	524,287	0.0933	314,111	263	3,953	1,602
20	2,097,151	2,097,151	0.3317	1,224,274	349	7,966	2,176
22	8,388,607	8,388,607	1.2369	4,814,578	402	16,142	2,310

For the average case ($T=50\%$ of the total time, which prunes part of the tree) the DFS visits 4,814,578 nodes on average at $n=22$, about 57% of its worst case, and the growth rate fitted between $n=10$ and $n=22$ is 1.99 per additional module: the average case is also exponential, with essentially the same base. Extrapolating the measured worst-case time (not measured directly), $n=30$ would take about 5 minutes and $n=40$ about 4 days on the same machine, which makes the exhaustive search impractical beyond a few dozen modules.

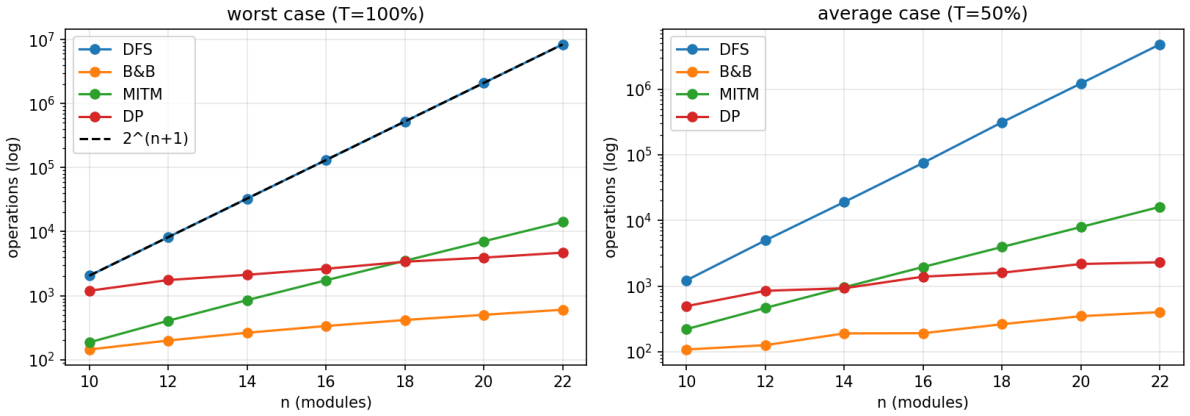


Figure 2: Operations versus n (log scale). The DFS follows $2^{n+1}-1$ in the worst case (left) and has the same exponential trend in the average case (right); DP, Branch and Bound and Meet-in-the-Middle grow much more slowly in this range.

In this range the DP is comparatively expensive in operations for small n (its cost is $n(T+1)$ regardless of the data), while Branch and Bound needs very few operations: even in the DFS worst case it performs only 602 operations at $n=22$, because when every module fits the greedy path is optimal and the bound prunes everything else. This scenario is therefore favorable to Branch and Bound; Section 6.7 shows that this does not hold for all instance families.

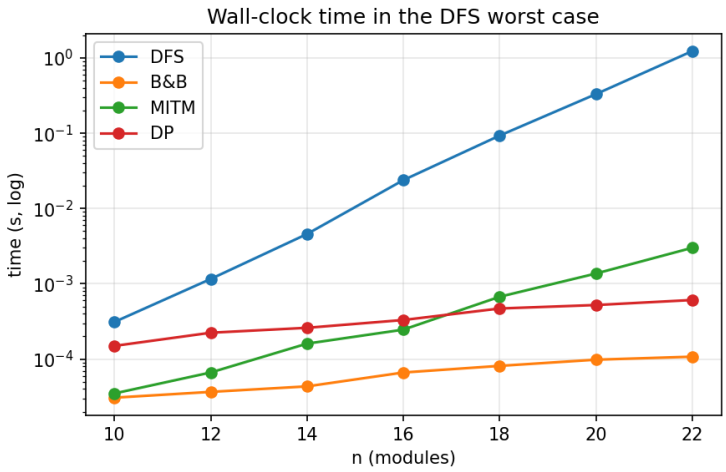


Figure 3: Wall-clock time versus n in the DFS worst case (log scale).

6.4 Dynamic programming and greedy at scale

The DFS cannot be run at large n, but the DP and the greedy can. Table 10 and Figure 4 report n from 50 to 600 modules (uncorrelated family, $\rho = 0.5$). The number of DP operations coincides *exactly* with $n(T+1)$ at every size, confirming Theorem 5. The DP solves an instance with 600 modules and $T = 3{,}060$ time units in about 0.37 s of pure Python, i.e. roughly 0.2 μs per cell. The vector version is about 40% faster because it avoids allocating the table. The greedy performs between 0.95 and 0.98 operations per $n \log_2 n$, as expected from Proposition 7, and takes less than two milliseconds.

Table 10: DP and greedy at scale (uncorrelated family, $R = 20$, $\rho = 0.5$).

n	T	DP ops $= n(T+1)$	DP time (s)	DP-vector (s)	Greedy ops	Greedy ops$/(n \log_2 n)$	Greedy (s)
50	273	13,700	0.0017	0.0010	269	0.95	0.0001
100	528	52,900	0.0081	0.0053	636	0.96	0.0002
200	1,097	219,600	0.0369	0.0247	1,486	0.97	0.0004
400	1,991	796,800	0.1396	0.0924	3,354	0.97	0.0011
600	3,060	1,836,600	0.3697	0.2230	5,400	0.98	0.0016

6.5 Pseudo-polynomial behavior

To demonstrate Proposition 6 we keep the same 18 modules and multiply all times and the budget by a factor from 1 to 1,024 (for instance, hours expressed in minutes and then in smaller units). The structure of the instance does not change, only the *magnitude* of T, which goes from 100 to 102,400. Table 11 and Figure 5 show that the DP operations grow in proportion to T (from 1,818 to 1,843,218, a factor of about 1,014 for a factor of 1,024 in T) and its time from 0.0002 s to 0.27 s, whereas the DFS, Branch and Bound, Meet-in-the-Middle and greedy counts do not change at all: they depend on n and not on T.

6.6 Quality of the greedy heuristic

Table 12 reports the mean and the worst gap of the greedy and of the greedy with the best single module over 30 instances per cell, for $\rho = 0.3$; Figure 6 shows the gap as a function of the budget and of n.

Three observations follow. First, the *mean* gap is small and decreases with n: with many modules that are

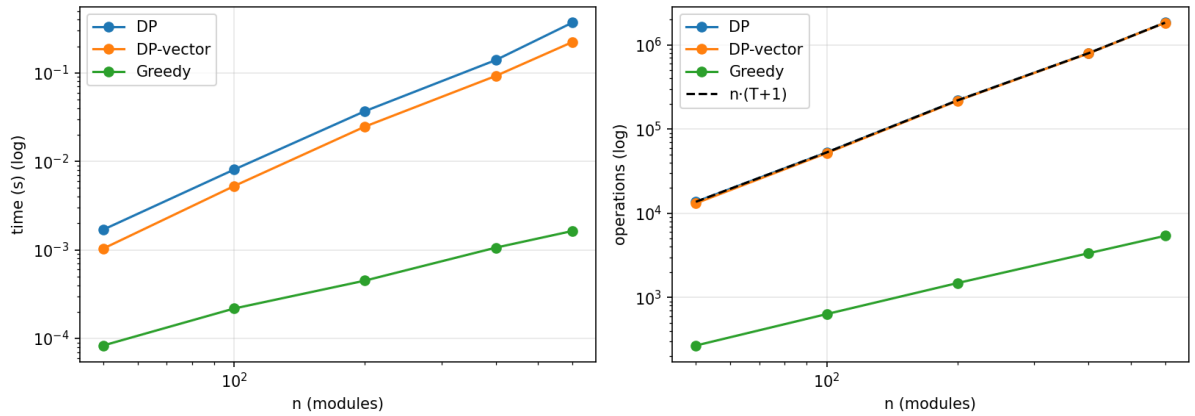


Figure 4: Time and operations of the DP and the greedy as n grows (log–log). The dashed line is $n(T+1)$, which the DP operation count matches exactly.

Table 11: Effect of the magnitude of T ($n = 18$ fixed; times and budget multiplied by the factor).

Factor	T	DP ops	DP time (s)	DFS ops	B&B ops	MITM ops	Greedy ops
$\times 1$	100	1,818	0.0002	313,216	265	4,005	74
$\times 4$	400	7,218	0.0009	313,216	265	4,005	74
$\times 16$	1,600	28,818	0.0038	313,216	265	4,005	74
$\times 64$	6,400	115,218	0.0153	313,216	265	4,005	74
$\times 256$	25,600	460,818	0.0662	313,216	265	4,005	74
$\times 1024$	102,400	1,843,218	0.2657	313,216	265	4,005	74

small relative to the budget, the greedy is nearly optimal (for instance, 0.8% on the strongly correlated family at $n = 60$). Second, the *worst* gap is much larger, especially with few modules (25.9% at $n = 8$ for $\rho = 0.3$, and 54.5% at $\rho = 0.1$, Table 13), and it reaches 99.8% on the adversarial instance of Section 4.3: a small average gap offers no guarantee. Third, at $\rho = 0.3$ the strongly correlated family yields the largest mean gap for every n tested, in agreement with the intuition that short modules with high ratio block longer modules that are worth more; the ordering is not universal, though (at $\rho = 0.1$ and $n = 15$ the weakly correlated family is slightly worse, 5.5% against 4.6%).

Table 13 shows what the variant of Proposition 9 does in practice. With tight budgets and few modules, where a single module nearly fills the knapsack, it repairs the worst cases (for $n = 8$, the mean gap on the weakly correlated family falls from 6.7% to 0.8% and the worst gap from 54.5% to 12.5%). For $n = 15$ it brings practically no improvement, and at $\rho = 0.3$ almost none for any n tested (Table 12), so the main benefit of the variant is the guarantee, not the average quality.

6.7 Exact algorithms across instance families

Table 14 and Figure 7 compare Branch and Bound, Meet-in-the-Middle and the DP on uncorrelated and strongly correlated instances with $R = 1000$ (so that the budget is large, which is the regime in which the DP is most expensive), $\rho = 0.5$, five instances per row.

DP and Meet-in-the-Middle cost the same in both families, as predicted: their cost depends on n and T (DP) or on n (MITM), not on the relation between times and scores. Branch and Bound, in contrast, depends strongly on the family: on strongly correlated instances it needs 11 to 15 times more operations than on uncorrelated ones (for instance 17,162 against 1,377 on average at $n = 40$), and its worst observed

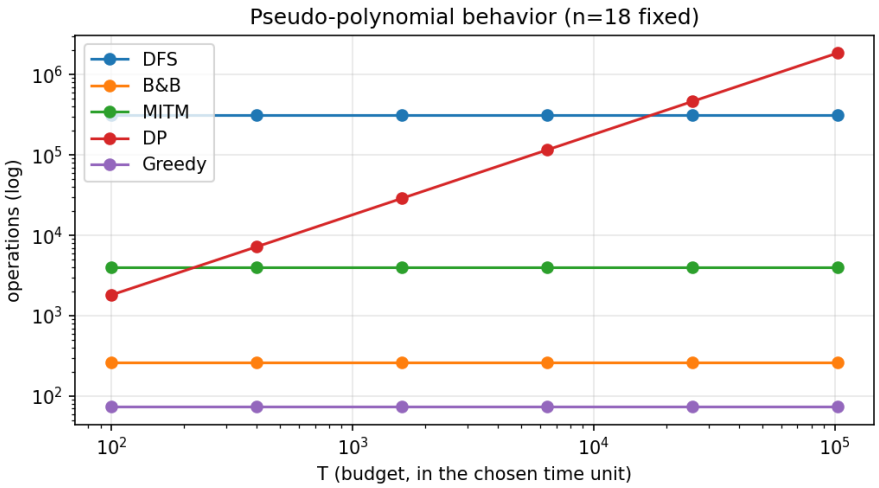


Figure 5: Operations versus T for fixed $n = 18$ (log–log). Only the DP grows with T.

Table 12: Gap with respect to the optimum, $\rho = 0.3$ (30 instances per row, $R = 20$).

n	Family	Greedy mean gap	Greedy max gap	Greedy+item mean	Greedy+item max
8	Uncorrelated	1.8%	10.8%	1.8%	10.8%
8	Weakly corr.	5.5%	25.9%	5.4%	24.0%
8	Strongly corr.	7.0%	19.0%	7.0%	19.0%
15	Uncorrelated	1.7%	9.1%	1.7%	9.1%
15	Weakly corr.	2.7%	8.9%	2.7%	8.9%
15	Strongly corr.	3.2%	7.9%	3.2%	7.9%
30	Uncorrelated	0.4%	2.8%	0.4%	2.8%
30	Weakly corr.	1.0%	3.6%	1.0%	3.6%
30	Strongly corr.	2.2%	4.6%	2.2%	4.6%
60	Uncorrelated	0.5%	1.8%	0.5%	1.8%
60	Weakly corr.	0.2%	0.9%	0.2%	0.9%
60	Strongly corr.	0.8%	2.2%	0.8%	2.2%

run (44,475 operations) is 21 times its worst uncorrelated run (2,087). This agrees with the literature, where strongly correlated instances are reported as hard for branch-and-bound algorithms [9]; a plausible explanation, which we did not test, is that the fractional bound is less effective on them. Meet-in-the-Middle grows by a factor of 32 from $n = 30$ to $n = 40$ (261,460 to 8,387,202), exactly the factor $2^{10/2}$ predicted by Theorem 11, and from $n = 30$ on it already costs more than the DP (21.5 times more at $n = 40$) because T is small relative to $2^{n/2}$.

6.8 FPTAS

Table 15 compares the FPTAS with the exact DP on a strongly correlated instance with $n = 30$.

The FPTAS reaches the optimum in every case (ratio 1.0000, far better than the guaranteed $1 - \varepsilon$), but at a high cost: it performs 7.8 times more operations than the exact DP with $\varepsilon = 0.5$ and 78 times more with $\varepsilon = 0.05$. The operation count grows in proportion to $1/\varepsilon$ (a factor of about 2.5, 2.0 and 2.0 as ε goes from 0.5 to 0.2, 0.1 and 0.05), in agreement with Theorem 12. With a small T, approximating is more expensive than solving exactly; the FPTAS pays off only when T is so large that $n(T+1)$ exceeds n^3/ε.

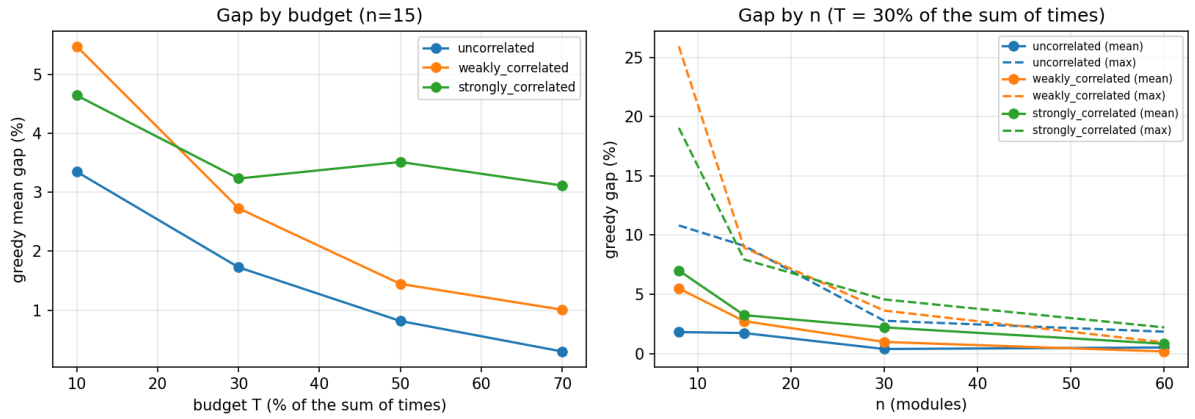


Figure 6: Mean gap of the greedy by budget ratio ($n = 15$, left) and mean and maximum gap by n ($\rho = 0.3$, right).

Table 13: Effect of adding the best single module to the greedy, $\rho = 0.1$ (30 instances per row).

n	Family	Greedy mean gap	Greedy max gap	Greedy+item mean	Greedy+item max
8	Uncorrelated	2.5%	50.0%	0.0%	0.0%
8	Weakly corr.	6.7%	54.5%	0.8%	12.5%
8	Strongly corr.	7.1%	35.0%	1.1%	11.1%
15	Uncorrelated	3.4%	23.1%	3.4%	23.1%
15	Weakly corr.	5.5%	19.0%	5.0%	19.0%
15	Strongly corr.	4.6%	14.6%	4.6%	14.6%

6.9 The article's DFS on a prerequisite graph

Finally, `dfs_paths` was executed on a complete DAG (an edge from i to every $j > i$), the worst case for the strategy of the article. Table 16 shows that the number of generated paths is exactly 2^{n-1} for every n tested (32,768 paths for $n = 16$), as derived in Section 3: the exhaustive path enumeration of the article is exponential in the size of the course graph.

7 Discussion

7.1 Which algorithm is the most suitable for learning-path selection?

Our analysis and experiments support the choice of **dynamic programming** as the most suitable algorithm for this scenario, for six reasons.

1. **The answer must be optimal.** A recommendation that wastes part of the learner's time is a poor recommendation. The greedy lost up to 54.5% of the score on small random instances, 15.4% on the test instance and 99.8% on the adversarial instance (Section 6.6); the DP never loses anything (Lemma 3).

2. **The parameter that makes the DP expensive is small in this domain.** The study load of a learner is expressed in minutes or blocks of a few minutes, so T stays in the low thousands. With 600 modules and $T = 3{,}060$ the DP took 0.37 s in pure Python (Table 10). The pseudo-polynomial nature (Proposition 6) becomes a problem only if time is measured in very fine units, as in the experiment of $T \approx 10^5$, where the cost reaches 1.8×10^6 operations for just 18 modules.

Table 14: Operations of the exact algorithms by family ($R = 1000$, $\rho = 0.5$, 5 instances per row).

Family	n	B&B mean	B&B max	MITM	DP
Uncorrelated	20	428	690	8,028	95,070
Uncorrelated	30	683	834	261,460	211,032
Uncorrelated	40	1,377	2,087	8,387,202	389,768
Strongly corr.	20	6,596	26,368	8,028	95,070
Strongly corr.	30	7,655	16,935	261,460	211,032
Strongly corr.	40	17,162	44,475	8,387,202	389,768

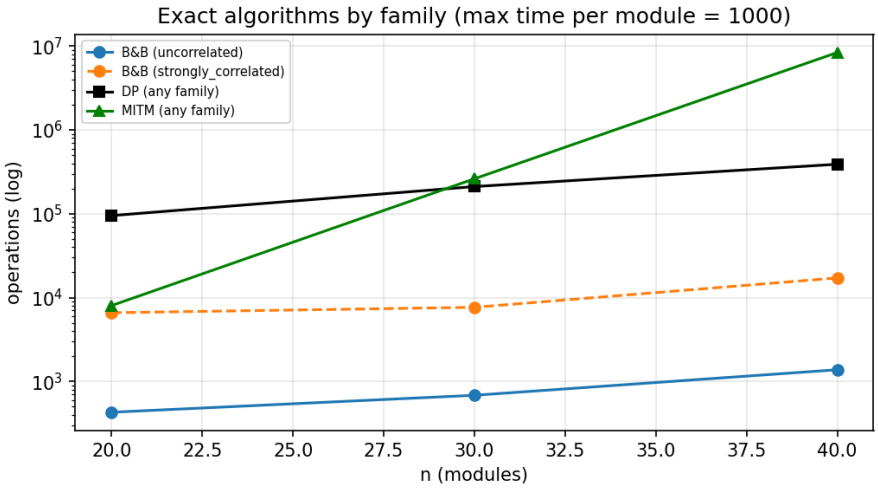


Figure 7: Mean operations by n for Branch and Bound on two families, compared with the DP and Meet-in-the-Middle (which do not depend on the family).

3. **The cost is predictable.** The DP performs exactly $n(T+1)$ operations on every instance (Theorem 5), whereas Branch and Bound varied between 1,377 and 44,475 operations at the same n depending on the family (Table 14). For a system that answers learners interactively, a worst-case guarantee is worth more than a better average.

4. **Realistic data are correlated.** Longer modules plausibly carry more content and a higher score. The strongly correlated family is exactly where the greedy loses the most on average and where Branch and Bound becomes 11 to 15 times more expensive, while the DP is unaffected.

5. **One table answers every budget.** Since $D[i][t]$ is defined for *at most* t time units, the last row gives the optimal score for any study load up to T, and the reconstruction works for each of them. No other algorithm studied provides this without being run again, and it answers the natural question "what if I only had eight hours?".

6. **It is simple to prove, to explain and to extend.** The optimal substructure is short (Lemma 3), the table is visual, and the reconstruction explains why each module was selected, which is valuable in an educational recommender.

7.2 When another algorithm wins

The DP is not the best algorithm in every regime, and the experiments show it. Table 17 summarizes which algorithm to choose depending on the situation.

Table 15: FPTAS versus exact DP ($n = 30$, strongly correlated, $R = 20$, $\rho = 0.5$).

Algorithm	ε	Score	Score / optimum	Operations
exact DP	—	365	1.0000	4,680
FPTAS	0.5	365	1.0000	36,690
FPTAS	0.2	365	1.0000	91,680
FPTAS	0.1	365	1.0000	183,330
FPTAS	0.05	365	1.0000	366,630

Table 16: Paths generated by the prerequisite-graph DFS on a complete DAG.

n	4	6	8	10	12	14	16
Paths generated	8	32	128	512	2,048	8,192	32,768
2^{n-1}	8	32	128	512	2,048	8,192	32,768

A point that deserves honesty: in our experiments Branch and Bound performed fewer operations than the DP for every instance size tested ($n \leq 40$), even on strongly correlated instances. We nevertheless consider the DP more suitable because Branch and Bound offers no polynomial guarantee, its cost fluctuates strongly with the data (and the literature reports instances on which it explodes [9]), and our largest instances have only 40 modules. If an application is known to produce easy instances with moderate n, Branch and Bound is a legitimate choice.

7.3 Agreement with the literature

The experiments reproduce, on our implementation, the main statements of the theory presented in Section 2:

- The DP is pseudo-polynomial: its operation count is exactly $n(T + 1)$ and grows with the magnitude of T alone [5, 8].
- Meet-in-the-Middle grows as $2^{n/2}$ and does not depend on T [10].
- Strongly correlated instances are harder for branch-and-bound algorithms, while DP is insensitive to the correlation [9].
- The ratio-based greedy has no constant guarantee, but the variant with the best single module guarantees one half of the optimum [8]; the FPTAS cost grows with $1/\varepsilon$ [11, 12].
- The exhaustive search of the article is exponential in the size of the graph (Section 6.9), which explains why a reasonable system either limits the size of the course graph or uses a more elaborate method [1].

7.4 Threats to validity

- **Synthetic instances.** Scores and times are generated by three families; no real course data were used. The conclusion about correlation rests on the hypothesis that longer modules score higher.
- **Strongly correlated constant.** We use $R/2$ instead of the $R/10$ common in the literature (Section 6.1), which may exaggerate the greedy gap relative to standard benchmarks.
- **Size of the instances.** The DFS was run up to $n = 22$ and Branch and Bound and Meet-in-the-Middle up to $n = 40$ (in the family experiment); results for larger n rely on the theory and on extrapolation.

Table 17: Choice of algorithm by regime.

Situation	Most suitable algorithm
Small integer T (minutes or hours), n up to thousands, an exact answer and a guarantee are required	Dynamic programming
Immediate answer, many modules that are small relative to T, a loss of 1 to 3% is acceptable	Greedy (preferably with the best single module, which provides the 1/2 guarantee)
Time measured in very fine units (T in the millions) and n up to about 40	Meet-in-the-Middle, whose cost does not depend on T
Both n and T enormous and an error ε is tolerated	FPTAS (in theory; at our scales it did not pay off)
Easy instances and moderate n, average speed matters more than a worst-case guarantee	Branch and Bound (in our tests it needed fewer operations than the DP for $n \leq 40$)
Prerequisites between modules (graph)	Path enumeration as in the article, or integer programming; tree-shaped precedence admits specific DPs; on general DAGs the problem is NP-hard [3]

- **Python and one machine.** Absolute times depend on the interpreter and hardware; conclusions rely on operation counters. The counters of different algorithms are not strictly commensurable (a node of Branch and Bound is more expensive than a cell of the DP), although we included the cost of the bound computation to reduce this bias.
- **Few instances per point.** Averages use 5 to 30 instances per point, so small differences should not be over-interpreted.

8 Members and Activities

Table 18: Activities carried out by each group member.

Member	Activities
[Member Name 1]	Study of the article (RUTICO) and of the course graph; implementation of `dfs.py` and `graph_dfs.py`; analysis of the DFS (Sections 3.1 and 6.9).
[Member Name 2]	Design and implementation of the dynamic programming (`dp.py`); proofs of correctness and complexity of the DP (Section 5); test instance and DP table.
[Member Name 3]	Implementation of the greedy algorithm and of the sort (`greedy.py`, `sorting.py`); extra algorithms (`branch_and_bound.py`, `meet_in_the_middle.py`, `fptas.py`); instance generators.
[Member Name 4]	Experimental routine (`experiment.py`), execution of the experiments and plots; writing and integration of the report.

All members reviewed the whole code and the whole report, and each one is able to explain every code segment and every complexity derivation in the presentation, as required by the assignment.

9 Conclusion

General comments. We studied the selection of learning modules under a study-time budget, modeled as the 0/1 knapsack problem, starting from the article RUTICO, whose solution is an exhaustive DFS. We showed that dynamic programming is the same search with memoization: the decision tree has 2^n leaves but only $n(T+1)$ distinct states. We proved the correctness and the best-, worst- and space

complexity of the DFS ($\Theta(n)$ to $\Theta(2^n)$ nodes, with exactly $2^{n+1}-1$ nodes when everything fits), of the DP ($\Theta(nT)$, pseudo-polynomial) and of the greedy ($\Theta(n \log n)$, without a constant guarantee unless the best single module is considered), and of three additional algorithms. The experiments, based on hardware-independent operation counters and validated on more than a thousand random instances, confirmed each of these results. The DP is the most suitable algorithm for the learning-path scenario because it is exact, has a cost that does not depend on the data, and the budget T is small in practice; the experiments also showed regimes in which Branch and Bound, Meet-in-the-Middle or the greedy are preferable.

Main difficulties.

- **The article does not use dynamic programming.** The solution of the authors is a DFS, so we had to implement it and justify the DP through its relation to the DFS (memoization) rather than assume it.
- **Bridging the article and the knapsack.** The article works on a graph with prerequisites; the knapsack ignores them. Deciding how to state this relaxation, and keeping a graph version (`graph_dfs.py`) for the authors' setting, required care.
- **Fair comparisons.** Comparing algorithms with different cost structures is delicate: a Branch and Bound node costs $\mathcal{O}(n)$ because of the bound, a DP cell costs $\mathcal{O}(1)$. We counted the bound steps explicitly and we did not rely on wall-clock time alone.
- **Interpreting the greedy.** Its mean gap is small for large n but its worst gap is large; reporting only the mean would have been misleading, and showing only the worst case would have been unfair. The families and the budget ratio were needed to tell the whole story.
- **Exponential experiments.** The DFS cannot be run beyond about 22 modules in reasonable time, so growth beyond that point is established by proof and extrapolation, not by measurement.
- **Implementation details.** Hand-writing the merge sort to keep $\Theta(n \log n)$ in every case, using integer arithmetic for ratio comparisons, and handling deep recursion in the memoized DP.

Limitations. The relaxation without prerequisites, synthetic instances, a single machine and interpreter, and instance sizes up to $n = 600$ (and $n = 40$ for the exponential algorithms) bound the scope of the conclusions (Section 7).

Future work. (i) Incorporate prerequisites: the precedence-constrained knapsack, starting from tree-shaped prerequisites, for which dynamic programming remains applicable. (ii) Several constraints at once (time and difficulty), i.e. a multidimensional knapsack, where the DP table acquires one dimension per constraint. (iii) Replace the synthetic scores by the time and score estimates of the article and real course data. (iv) Evaluate larger instances with a compiled implementation, and mixed strategies such as DP for the bulk of the budget and Branch and Bound for the remainder.

References

[1] A. H. Nabizadeh, A. M. Jorge, and J. P. Leal. RUTICO: Recommending Successful Learning Paths Under Time Constraints. In *Adjunct Publication of the 25th Conference on User Modeling, Adaptation and Personalization (UMAP '17)*. ACM, 2017. DOI: 10.1145/3099023.3099035.

[2] A. H. Nabizadeh, J. P. Leal, H. N. Rafsanjani, and R. R. Shah. Learning path personalization and recommendation methods: A survey of the state-of-the-art. *Expert Systems with Applications*, 159:113596, 2020. DOI: 10.1016/j.eswa.2020.113596.

[3] N. Samphaiboon and T. Yamada. Heuristic and exact algorithms for the precedence-constrained knapsack problem. *Journal of Optimization Theory and Applications*, 105(3):659–676, 2000.

[4] R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.

[5] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[6] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[7] G. B. Dantzig. Discrete-variable extremum problems. *Operations Research*, 5(2):266–288, 1957.

[8] H. Kellerer, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer, 2004.

[9] D. Pisinger. Where are the hard knapsack problems? *Computers & Operations Research*, 32(9):2271–2284, 2005.

[10] E. Horowitz and S. Sahni. Computing partitions with applications to the knapsack problem. *Journal of the ACM*, 21(2):277–292, 1974.

[11] O. H. Ibarra and C. E. Kim. Fast approximation algorithms for the knapsack and sum of subset problems. *Journal of the ACM*, 22(4):463–468, 1975.

[12] E. L. Lawler. Fast approximation algorithms for knapsack problems. *Mathematics of Operations Research*, 4(4):339–356, 1979.

[13] A. H. Land and A. G. Doig. An automatic method of solving discrete programming problems. *Econometrica*, 28(3):497–520, 1960.

[14] P. J. Kolesar. A branch and bound algorithm for the knapsack problem. *Management Science*, 13(9):723–735, 1967.

[15] S. W. Golomb and L. D. Baumert. Backtrack programming. *Journal of the ACM*, 12(4):516–524, 1965.

[16] R. E. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[17] D. Michie. "Memo" functions and machine learning. *Nature*, 218:19–22, 1968.

[18] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 4th edition, 2022.

[19] J. Kleinberg and É. Tardos. *Algorithm Design*. Addison-Wesley, 2006.

[20] D. E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.

[21] V. V. Vazirani. *Approximation Algorithms*. Springer, 2001.

[22] D. P. Williamson and D. B. Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011.

A Reproducing the Results

Requirements. Python 3.10 or newer and matplotlib (only for the plots), on Linux:

```
pip install matplotlib
python experiment.py                          # all blocks, about 1 minute
python experiment.py --quick                  # reduced version
python experiment.py --only demo correctness
```

The output of a full run is stored in `results/full_output.txt`; the plots and the CSV files used by this report are in `results/`. The tables of Section 6 are regenerated from the CSV files with `python report/make_tables.py`, and the report is compiled with `pdflatex report.tex` (twice).

Experiment blocks. Table 19 maps each block of `experiment.py` to its parameters, outputs and the section of this report that discusses it. The operation counts are identical in every run; wall-clock times vary with the machine.

Table 19: Experiment blocks, parameters and outputs.

Block	Parameters (full run)	Outputs	Section
`demo`	Test instance, $T = 10$; adversarial instance, $T = 1000$	standard output	4.3
`correctness`	1,200 random instances, $n \leq 16$, seed 2024, $R \in \{5, 20\}$, $\rho \in \{0.1, \ldots, 1.0\}$	standard output	6.2
`time`	$n = 10, 12, \ldots, 22$; worst case: uncorrelated, $\rho = 1.0$; average: 5 instances, $\rho = 0.5$	`exponential_time.csv`, `fig_operations_vs_n.png`, `fig_time_vs_n.png`	6.3
`scale`	$n \in \{50, 100, 200, 400, 600\}$, uncorrelated, $\rho = 0.5$	`scale.csv`, `fig_scale.png`	6.4
`quality`	$n \in \{8, 15, 30, 60\}$, 30 instances per point, $\rho \in \{0.1, 0.3, 0.5, 0.7\}$, three families	`greedy_quality.csv`, `fig_greedy_quality.png`	6.6
`pseudo`	$n = 18$, factors $1, 4, \ldots, 1024$, uncorrelated, $\rho = 0.5$	`pseudopolynomial.csv`, `fig_pseudopolynomial.png`	6.5
`families`	$n \in \{20, 30, 40\}$, 5 instances, $R = 1000$, $\rho = 0.5$, uncorrelated and strongly correlated	`exact_by_family.csv`, `fig_exact_by_family.png`	6.7
`fptas`	$n = 30$, strongly correlated, $\varepsilon \in \{0.5, 0.2, 0.1, 0.05\}$	`fptas.csv`	6.8
`graph`	complete DAG, $n = 4, 6, \ldots, 16$	`graph_dfs.csv`	6.9